

Advanced Manual **Smart Contract Audit**

September 30, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

Nlyra

Real Yield Staking v2, RealYieldStaking.sol

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	3	0	2	1
● Low-Risk	4	0	4	0
● Medium-Risk	0	0	0	0
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

Nlyra

Website	nlyra.xyz
---------	-----------

Blockchain	Robinhood Chain (Arbitrum Orbit, chainId 4663)
------------	--

Source Code	github.com/NlyraProject/nlyra-platform, staking-v2/src (commit 19880c0)/RealYieldStaking.sol (1,140 lines)
-------------	--

Contract Address	0x5CB0Cb16cA019bcff4E494b32575848E4CdB5aF8
------------------	--

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Audit Summary	2
Audit Scope	3
Audit Method	3
Automated Vulnerability Check	3
Manual Code Review	3
Used tools	3
Table of Contents	4

● All Documentation Inside The Contract Is Written In Spanish

Informational - Does not compromise the functionality of the contract in any way **[Status: Resolved]**

Error Code	Description
RYS-I1	The explanatory comments inside the Solidity sources are written in Spanish, which originally narrowed the set of people able to independently review some genuinely subtle accounting. The team has published an English companion covering every explanatory note in all four audited files, in code order and with line numbers. We checked it against the repository rather than taking it on trust: it is complete across the four contracts, and the .sol files are untouched, so they still match the sources verified for the deployed contracts at commit 19880c0.

Code Snippet

```
// published alongside the code, verified by us against the repo:
//   staking-v2/CONTRACT-NOTES.md   (435 lines, all four contracts)

// and the sources themselves are unchanged, so verification holds:
//   git diff 19880c0 HEAD -- staking-v2/src/   ->   no changes
```

Recommendation

No further action needed. Keep the English notes in step with the code if the contracts are ever redeployed, since the only thing linking the two is the commit reference in the document.

● The Contract Is Close To The Maximum Size A Contract Can Be

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
RYS-I2	Compiled with the deployment settings, the contract comes to roughly 21,500 bytes against a network limit of 24,576, so about 88 percent of the space available. Nothing is wrong today, but there is little room left to add features to this contract later, which is worth knowing before a future change rather than during one.

Code Snippet

```
// compiled with the deployment settings
// (solc 0.8.24, optimizer 200 runs, viaIR, cancun):
//
//   RealYieldStaking  runtime size  21,527 bytes
//   network limit      24,576 bytes
//   headroom remaining    3,049 bytes (12%)
```

Recommendation

Treat the remaining space as a planning constraint. If more functionality is expected, decide early whether it belongs in this contract or in a separate one, rather than discovering the limit part way through a future change.

● What The Owner Can Do Is Tightly Limited, With One Small Gap

Informational - Does not compromise the functionality of the contract in any way **[Status: Acknowledged]**

Error Code	Description
RYS-I3	The owner cannot move staked funds, take rewards, change rates, tiers or cooldowns, or alter anyone's position. Pausing is capped at 30 days, expires on its own, cannot be renewed for another 30 days, and never blocks claiming or withdrawing. This is a well constrained design and we want it on the record. The one small gap is that the rescue function can move USDG, which the contract only ever holds for a few moments inside a single transaction.

Code Snippet

```
// blocked from rescue: the staked token and the reward token
if (token == address(NLYRA) || token == address(WETH))
    revert NotRecoverable();
// USDG is not blocked, but is only held mid-transaction

// pausing is bounded and self-expiring
uint256 public constant MAX_PAUSE = 30 days;
uint256 public constant PAUSE_GAP = 30 days;
```

Recommendation

No change is needed. If the team wants to close the last gap, adding USDG to the list of tokens the rescue function refuses would cost nothing and would let you state plainly that the owner cannot touch any token involved in the product.

● Reward Conversions Have No Time Limit And Rely Only On The Figure The User Supplies

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
RYS-L1	When rewards are converted into a single token, the only protection against a bad price is a minimum amount the user supplies, and the request never expires. A transaction left pending can therefore execute much later at a price the user never agreed to, and a user who passes a minimum of zero, which interfaces often do by default, accepts whatever the pool gives them and can be sandwiched at their own expense. The contract and other users are not at risk.

Code Snippet

```
// no deadline parameter on either entry point
function claim(OutMode mode, uint256 minOut) external
function compound(uint256 minOut, uint8 tier, uint256 positionId)

// the swap runs to the end of the price range, so the pool
// itself imposes no limit:
zeroForOne ? MIN_SQRT_RATIO_P1 : MAX_SQRT_RATIO_M1

// the only guard is the caller's own figure:
if (out < minOut) revert Slippage(out, minOut);
```

Recommendation

Have the interface always calculate and fill in a sensible minimum rather than leaving it at zero, and warn users before submitting a conversion with no floor. Adding an expiry time to these two functions would remove the stale transaction problem entirely, and is worth considering for any future version.

● A Pause Freezes Compounding And All Position Sales For Up To 30 Days

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
RYS-L2	Pausing is a single switch that stops new stakes, lock extensions, compounding, unstake cancellations and all position transfers, which also halts every sale on the marketplace. Exits stay open so nobody is trapped, which is the right priority, but a pause aimed at one problem switches off unrelated features for up to 30 days and a seller with a live listing simply cannot sell during that window.

Code Snippet

```
// one switch covers all of these:
stake / stakeLocked / extendLock / cancelUnstake
compound / fundBonusReserve
offerPosition / acceptPosition / acceptPositionTo

// and therefore, indirectly, every sale on PositionMarket

// exits stay open, which is correct:
// claim, requestUnstake, withdrawLocked, withdraw, kick
```

Recommendation

Keep the design, which errs in the right direction, but treat a pause as a serious step with a clear plan for lifting it rather than something to be left running for the full 30 days. Tell users in advance what a pause stops and what it does not, so a seller with an active listing understands what happens. If a future version needs finer control, separating the pause on new deposits from the pause on transfers would let you respond to a problem in one area without freezing the other.

● Small Reward Batches Can Quietly Lose Their Eligibility For The Compound Bonus

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status:** **Acknowledged**]

Error Code	Description
RYS-L3	Rewards arriving from the project's own fee splitter are tagged as eligible for the five percent compounding bonus, but if the amount handed over is too small to open a reward batch, the contract keeps the money and stores the tag nowhere. It is still paid out later, so nothing is lost outright, but by then it counts as ordinary rewards and no longer earns the bonus. The sums are small by definition, since this only happens below the minimum threshold.

Code Snippet

```
// if both sides are below the minimum, nothing is opened
if ((add[I_WETH] < MIN_NOTIFY_WETH
    && add[I_NLYRA] < MIN_NOTIFY_NLYRA) || g.count == MAX_TRANCHES) {
    emit NotifySkipped(add[I_WETH], add[I_NLYRA]);
    return false; // the eligible tag is not carried forward
}

// on the next handover only the NEW amount is tagged:
uint256 ew = Math.min(Math.min(wethIn, add[I_WETH]) / REWARD_DURATION, rw);
```

Recommendation

Carry the untagged eligible amount forward so it keeps its status until it is actually paid into a batch, rather than letting it lapse. If that is judged not worth the extra complexity for the small sums involved, note the behaviour publicly so nobody is surprised that a bonus figure moved, and keep the splitter's minimum transfer comfortably above the threshold so the situation rarely arises.

● Anyone Can Push Unwanted Locks Onto A Wallet And Temporarily Fill Its Position Slots

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
RYS-M1	A locked position is handed to another wallet in two steps, and the function used to accept it also lets the accepting address forward it to somebody else, which is how the marketplace delivers a purchase. The final recipient is never asked, so anyone can be given a position without agreeing to it, and because each wallet holds at most 32 positions an attacker using two wallets can fill every slot with worthless locks and stop the target opening a lock or buying on the marketplace. We originally rated this Medium on the understanding that the target had to wait for each unwanted lock to expire before clearing it. That was wrong, and we have corrected it: we tested the alternative and an unwanted lock can be moved straight out to another wallet using the same two step transfer, which frees the slot in the same block and costs the target fewer transactions than filling costs the attacker. Funds, rewards, claims and withdrawals are never affected, so what remains is a temporary nuisance the target can undo at will, and we have reclassified it as Low.

Code Snippet

```
// the accepting address chooses the final recipient, who is
// never consulted:
function acceptPositionTo(address from, uint256 positionId,
                        address recipient) external ... {

// but the SAME two step transfer is also the way out, with no
// wait for the lock to expire:
//   victim.offerPosition(slot, ownSecondWallet)
//   ownSecondWallet.acceptPosition(victim, slot)

// verified both directions against the real contract:
// [PASS] test_poc_unsolicitedLocksExhaustVictimSlots()
```

```
// [PASS] test_claim_unwantedLockCanBeMovedOutImmediately()  
// [PASS] test_claim_clearingIsCheaperThanAttacking()
```

Recommendation

Keep the recipient opt in planned for the next version, since requiring agreement before a position can be delivered is the clean fix. Until then the app side measures described, a slot counter, detection of tiny locks and a one click move or removal, are the right mitigation, and it is worth confirming the move option stays reachable for a wallet whose slots are already full. One edge is worth knowing: the escape route uses the same transfer functions a pause blocks, so during a pause an affected wallet cannot clear its slots, although it could not open a lock during a pause either.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.